

User:

Its main tasks would be as an exec assistant, social media marketing and management, research and development, and being able to operate my IDE

Kimi:

Based on your requirements for an executive assistant, social media manager, R&D researcher, and IDE operator, you’re looking to build a sophisticated **AI agent system** rather than a simple chatbot. This requires multiple specialized agents working together. Let me search for the most current approaches to building this type of multi-agent AI asset. Based on your requirements for an executive assistant, social media manager, R&D researcher, and IDE operator, you’re looking to build a **multi-agent AI workforce**—one of the defining trends of 2025 . This requires specialized agents working together through an orchestration layer. Here’s how to architect this:

Your Multi-Agent AI Architecture

Instead of building one monolithic AI, you’ll deploy **4 specialized agents** coordinated by a central orchestrator:

Agent	Core Function	Key Tools/Integrations
Executive Assistant Agent	Calendar, email triage, meeting prep, task prioritization	Google Workspace/Outlook, Slack, CRM, scheduling APIs
Social Media Manager Agent	Content creation, scheduling, engagement, analytics	Instagram/Twitter/LinkedIn APIs, Canva/Figma, analytics platforms
R&D Research Agent	Market analysis, trend monitoring, competitive intel, summarization	Academic databases, web scraping, news APIs, vector databases

Agent	Core Function	Key Tools/Integrations
IDE	Code generation,	VS Code/JetBrains, GitHub, terminal/
Operator	debugging,	CLI, cloud sandboxes
Agent	refactoring, PR management	

Build Strategy: Two Approaches

Approach A: Integrated Platform (Faster, Less Flexible)

Use existing multi-agent platforms that handle orchestration for you:

- **Relevance AI** or **Lindy**: Pre-built agent templates for each role with workflow automation
- **Microsoft Copilot Studio / Power Platform**: Deep enterprise integration, can connect to IDE via extensions
- **GitHub Agent HQ**: Already lets you run multiple coding agents (Claude, Codex, Copilot) in one workspace

Pros: 80% faster setup, built-in security, managed infrastructure

Cons: Limited customization, subscription costs scale with usage, vendor lock-in

Approach B: Custom Multi-Agent System (More Control, Longer Build)

Build your own orchestration layer using **AI agent frameworks**:

1. **Orchestration Engine**: Use frameworks like **AutoGen**, **CrewAI**, or **LangGraph** to coordinate agents
2. **Shared Memory**: Vector database (Pinecone/Weaviate) for cross-agent knowledge
3. **Tool Integration**: MCP (Model Context Protocol) servers for standardized tool access
4. **IDE Control**: Claude Code CLI or OpenAI Codex CLI integrated via subprocess calls or APIs

Pros: Full customization, own your IP, cost-effective at scale

Cons: Requires Python/TypeScript development, infrastructure management

Critical Component: The IDE Operator Agent

For IDE control specifically, you have two leading options in 2026:

Claude Code (Anthropic): - Runs locally in terminal with deep codebase awareness - Interactive, developer-in-the-loop approach—ideal for complex refactoring - Uses agentic search to understand entire codebase without manual context selection - Best for: Complex multi-file edits, local development workflows, maintaining control

OpenAI Codex Agent: - Cloud-based autonomous execution in isolated sandboxes - Can work asynchronously, generates PRs automatically - Now integrated into VS Code and JetBrains IDEs - Best for: Delegating end-to-end tasks, parallel processing, GitHub-centric workflows

Recommendation: Use both—Claude Code for deep local development, Codex for autonomous task delegation and code review . GitHub’s Agent HQ now lets you run both in the same workspace .

Implementation Roadmap

Phase 1: Foundation (Weeks 1-2)

1. **Set up your “Agent HQ”:** Use GitHub Agent HQ or similar to get Claude and Codex working in your IDE
2. **Build the Executive Assistant first:** It’s the highest-value, lowest-risk starting point. Use Lindy or Motion for calendar/email automation
3. **Establish knowledge base:** Centralize your documents, brand guidelines, and code documentation in a vector store

Phase 2: Specialization (Weeks 3-4)

4. **Deploy Social Media Agent:** Connect to Buffer/Hootsuite APIs or native platform APIs for content scheduling
5. **Activate R&D Agent:** Configure web scraping + academic paper monitoring (arXiv, Google Scholar APIs)
6. **Integrate IDE Agent:** Connect Claude Code/Codex to your orchestration layer via CLI commands or MCP servers

Phase 3: Orchestration (Weeks 5-6)

7. **Build the “Manager Agent”:** An LLM that routes tasks to the right specialist agent

- Input: Natural language request (“Prepare Q3 report and tweet the highlights”)
 - Output: Task decomposition + agent assignment
8. **Create shared context:** Ensure agents can access each other’s outputs (research feeds content, code changes trigger documentation updates)
 9. **Add human-in-the-loop gates:** Critical decisions (code merges, public posts, calendar changes over X hours) require approval

Technical Stack Recommendation

Layer	Technology	Purpose
Agent Framework	CrewAI or LangGraph	Multi-agent orchestration, task routing
IDE Integration	Claude Code CLI + Codex CLI	Code generation, terminal commands
Memory/ Knowledge	Pinecone + Supabase	Vector search, structured data
Workflow Automation	n8n or Make	Connecting non-AI tools (CRM, email)
Deployment	Local + Cloud hybrid	Sensitive work local, heavy processing cloud

Key Success Factors

1. **Start narrow, expand wide:** Get the Executive Assistant working perfectly before adding social media. Multi-agent systems fail when individual agents are unreliable
2. **Own your context:** The value isn’t the LLM (commoditized)—it’s your proprietary data, carefully engineered prompts, and workflow integrations . Invest heavily in your knowledge base.

3. **Plan for agent-to-agent handoffs:** Define clear output formats so your R&D agent's research automatically feeds into your Social Media agent's content calendar and your Executive Assistant's briefing docs.
4. **Security architecture:** Your IDE agent has terminal access—implement permission systems. Claude Code's permission prompts can be bypassed with `--dangerously-skip-permissions` but that's risky . Consider running IDE agents in isolated containers.
5. **Cost management:** Codex appears more cost-efficient than Claude for heavy usage—GPT-5 runs roughly half the cost of Claude Sonnet with comparable quality . Budget \$100-300/month for multi-agent operations at your scale.

Would you like me to dive deeper into any specific agent (especially the IDE operator setup), or help you evaluate specific platforms for your use case?

Adding a Music Production Agent for mixing and mastering is an excellent extension to your AI workforce. This is a specialized domain requiring audio-specific AI models and professional DAW integration. Let me search for the current state of AI music production agents in 2025-2026.

Adding a Music Production Agent for mixing and mastering is a powerful extension to your AI workforce. This requires specialized audio AI models and deep DAW integration. Here's how to architect this fifth agent alongside your existing executive, social media, R&D, and IDE operators:

Your Music Production Agent Architecture

Agent Role	Primary Function	Key AI Tools	DAW Integration
Composition/Arrangement	Generate MIDI, chord progressions, instrumentation	MIDI Agent, Suno, Udio, AIVA	Logic Pro (AU), Ableton (VST3)
Mixing Agent	Balance levels, EQ, panning, dynamics, effects	iZotope Neutron, LANDR, CloudBounce	Real-time plugin processing

Mastering Agent Final polish, loudness optimization, format delivery

LANDR Mastering Plugin, iZotope Ozone 12, Apple Mastering Assistant

Master bus processing, automated export

Critical Decision: Cloud vs. Edge Processing

Cloud-based mastering (LANDR, eMastered, CloudBounce):

- Upload mix → AI analyzes → download master
- Fast, consistent, requires internet
- Limited control: choose "Warm/Balanced/Punchy" styles

Edge/on-device mastering (Ozone 12, Logic Pro Mastering Assistant):

- Real-time processing within your DAW
- Full parameter control, no latency, privacy preserved
- Ozone 12's Master Assistant builds custom chains you can edit; Stem Focus lets you rebalance vocals/drums/bass from a stereo file

Recommendation for your agent: Use hybrid approach—LANDR plugin for real-time mix feedback during production, Ozone 12 for final surgical mastering .

Technical Implementation

1. DAW Integration Layer

Your Music Production Agent needs direct DAW control. Current 2025-2026 capabilities:

- Logic Pro: Native AU plugin support + Apple Mastering Assistant (on-device AI)
- Ableton Live: VST3 plugin support, excellent for live/performance workflows
- Pro Tools: AAX format, industry standard for professional studios
- Universal: MIDI Agent connects ChatGPT/Claude directly to generate MIDI in any AU/VST3-compatible DAW

2. AI Mastering Stack (2026 State-of-the-Art)

Tool	Best For	Key AI Feature	Cost
------	----------	----------------	------

iZotope Ozone 12	Surgical control, problematic mixes	Stem Focus (AI source separation from stereo), Master Assistant	\$55-499 perpetual
------------------	-------------------------------------	---	--------------------

LANDR Mastering Plugin	Speed, real-time workflow	Real-time AI analysis while mixing	\$15.99/month
------------------------	---------------------------	------------------------------------	---------------

Apple Mastering Assistant Privacy, on-device Runs on M-series Neural Engine,
no cloud upload Included in Logic Pro

Waves Online Mastering Reference track matching Upload commercial track
to guide AI sonic profile Credit-based

3. Building the Agent Orchestration

Your Music Production Agent needs to coordinate with your other agents:

Workflow Example:

4. R&D Agent identifies trending sonic aesthetics from market research
5. Music Production Agent generates composition drafts (MIDI Agent + Suno/
Udio for inspiration)
6. Executive Assistant Agent schedules review sessions and deadlines
7. Social Media Agent prepares teaser content from work-in-progress stems
8. IDE Operator Agent manages version control for project files and automated
deployment pipelines

Integration Architecture:

User Request → Manager Agent → Task Decomposition

↓

Music Production Agent ↔ DAW (Logic/Ableton/Pro Tools)

↓

[Composition] MIDI Agent/Suno → Generate MIDI/audio

[Mixing] iZotope Neutron/LANDR → Balance/processing

[Mastering] Ozone 12/LANDR → Final polish

↓

Output → Shared Knowledge Base → Other Agents notified

Implementation Roadmap

Phase 1: Foundation (Week 1)

- Install MIDI Agent in your primary DAW (Logic Pro recommended for AI
integration)
- Connect API keys: OpenAI, Anthropic, or local LLMs (Ollama/LM Studio for
offline work)

- Test basic workflow: Natural language prompt → MIDI generation → instrument assignment

Phase 2: Mixing/Mastering Integration (Week 2)

- Add LANDR Mastering Plugin for real-time mix feedback
- Configure Ozone 12 for final mastering with Master Assistant as starting point
- Establish export pipeline: DAW → AI mastering → delivery formats (streaming, vinyl, spatial audio)

Phase 3: Multi-Agent Orchestration (Week 3-4)

- Build DAW control interface: Use AppleScript (Logic), Max for Live (Ableton), or ReaScript (Reaper) to allow your Manager Agent to execute DAW commands
- Create feedback loops: Mastering Agent reports back to R&D Agent on sonic trends; Social Media Agent auto-generates content from approved masters
- Human-in-the-loop gates: Final approval before publishing, budget thresholds for cloud processing

Key Technical Considerations

Audio Quality Standards:

- Export mixes at 24-bit/44.1kHz or 48kHz with -6dB to -3dB headroom for optimal AI mastering results
- Never submit MP3s for mastering—use lossless WAV/AIFF

Cost Management:

- Cloud mastering: ~\$25/month unlimited (LANDR Studio Pro)
- Perpetual plugins: Ozone 12 Advanced \$499 one-time
- API costs for MIDI generation: varies by LLM usage

Future-Proofing:

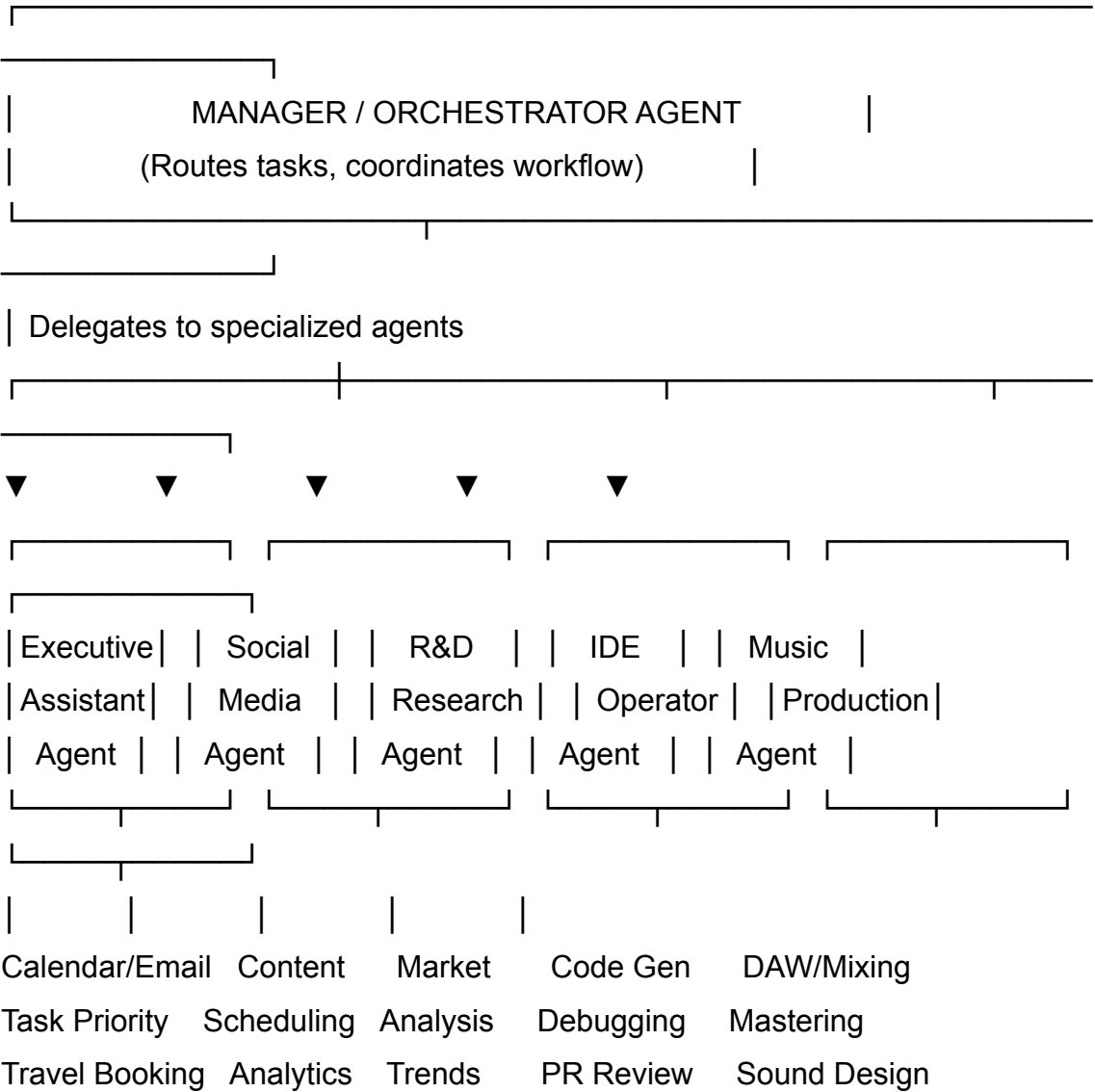
- Spatial/immersive audio: AI upmixing from stereo to Dolby Atmos is emerging (Masterchannel, Apple)

- Edge AI: On-device processing is becoming standard for privacy and latency

Would you like me to detail the specific DAW automation scripting (e.g., how your Manager Agent can trigger Logic Pro actions), or dive deeper into the mixing vs. mastering agent specialization?

Yes, exactly. You're building a Multi-Agent AI System—a coordinated workforce where each agent specializes in one domain but operates under unified orchestration. This is the dominant architecture for sophisticated AI implementations in 2026.

Your System Architecture



How They Work Together

Shared Memory Layer

All agents access a common knowledge base (vector database):

- Executive Agent knows your schedule when Music Agent plans studio time
- R&D Agent's market research feeds Social Media Agent's content calendar

- IDE Agent's commit messages trigger Executive Agent's project updates

Inter-Agent Communication

Agents don't just report to you—they report to each other:

- Music Agent finishes a master → notifies Social Media Agent to prepare release campaign
- R&D Agent identifies trending tech → IDE Agent suggests relevant code updates
- Executive Agent reschedules a meeting → Music Agent adjusts studio booking

Human-in-the-Loop Gates

Critical decisions route through you:

- Code merges (IDE Agent proposes, you approve)
- Public social posts (Social Agent drafts, you review)
- Calendar changes >2 hours (Executive Agent requests confirmation)
- Final masters (Music Agent delivers, you approve for distribution)

Implementation Approaches

Approach	Best For	Tools	Trade-off
Unified Platform	Speed, simplicity	Relevance AI, Lindy, Agent HQ	Less customization, vendor dependency
Modular Framework	Control, scalability	CrewAI, LangGraph, AutoGen	Requires development, you own the architecture
Hybrid	Best of both	Platform for some agents, custom for others	Complexity in integration

Why This Architecture Wins

1. Specialization > Generalization: A single agent trying to do everything performs worse than specialists coordinated by a manager
2. Failure Isolation: If your Music Agent's DAW crashes, your Executive Agent still manages your calendar
3. Parallel Processing: Multiple agents work simultaneously—R&D researches while Music mixes while Social schedules
4. Modular Upgrades: Swap out Claude Code for Codex in your IDE Agent without touching your Music Agent's LANDR integration

5. Cost Optimization: Use GPT-5 for simple tasks (Social Media), Claude for complex reasoning (R&D), local models for sensitive work (Executive)

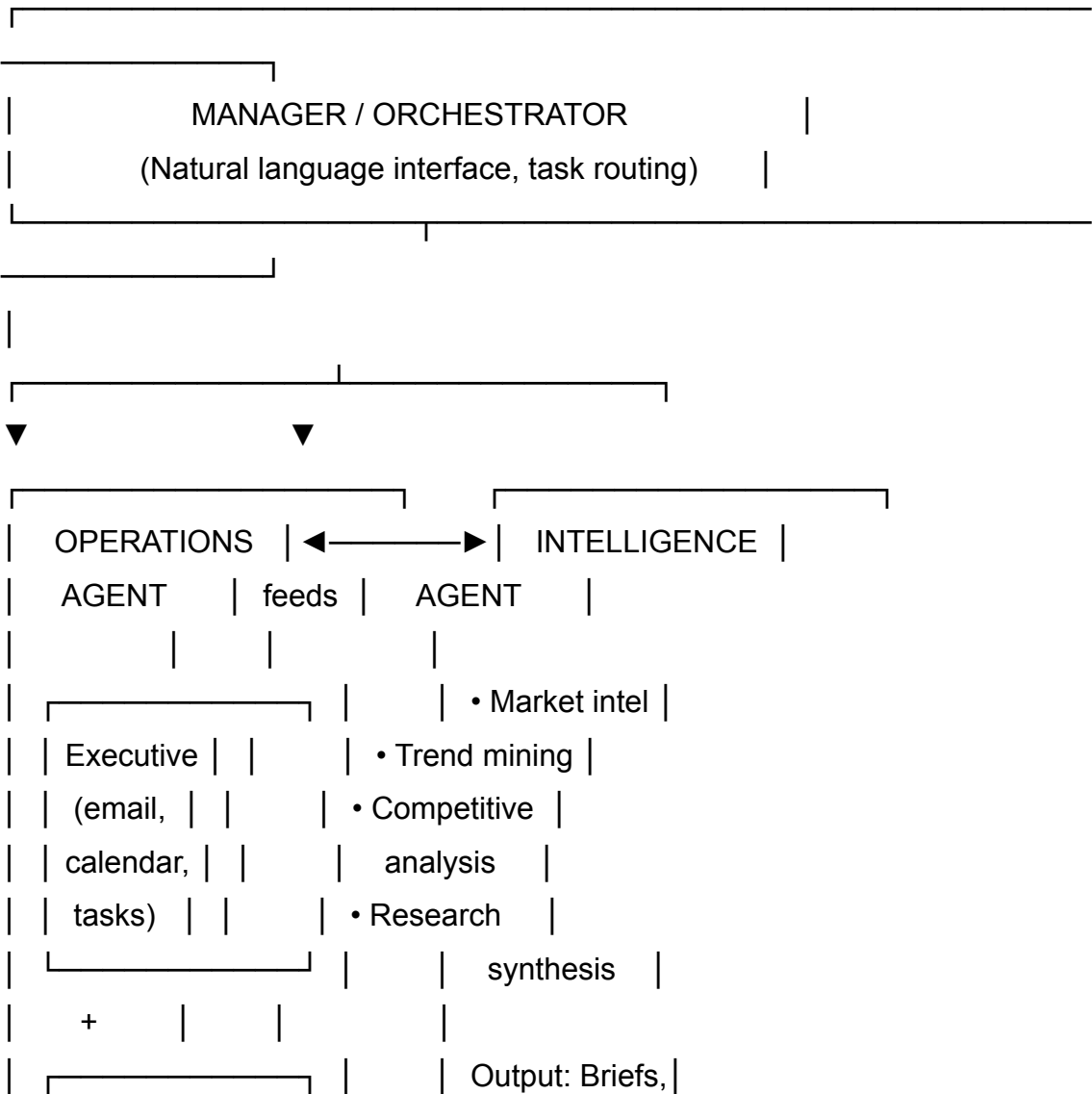
Your Next Step

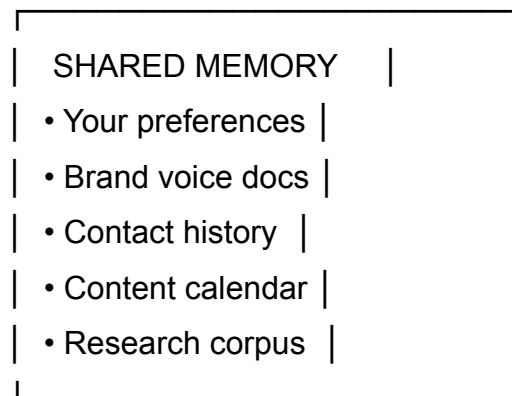
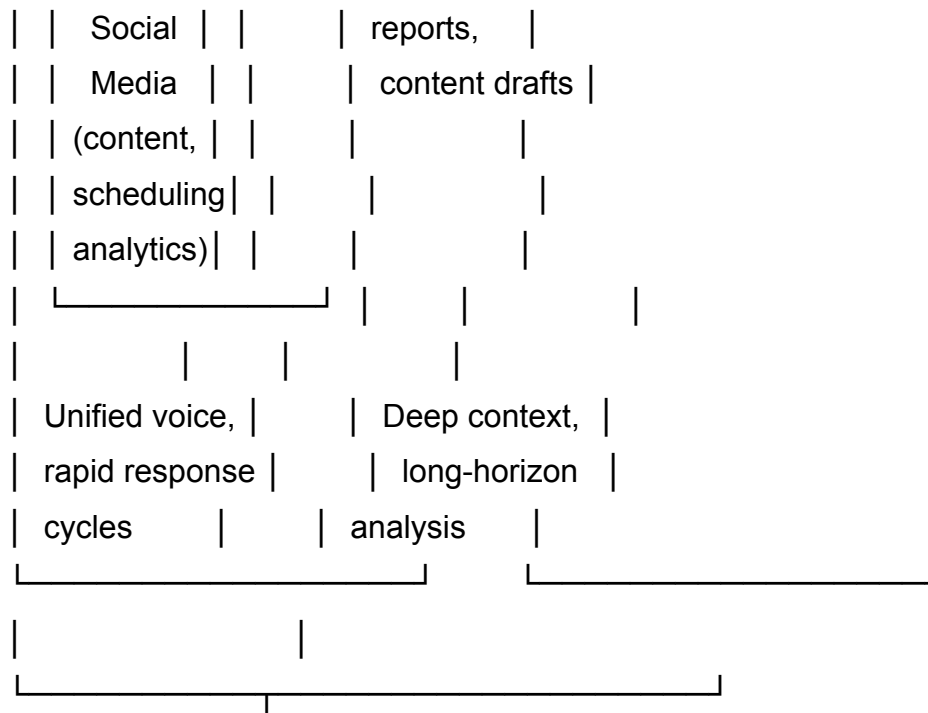
Start with two agents max—Executive + one other. Get the orchestration working before adding the full workforce. Most failed multi-agent systems result from trying to coordinate too many unreliable components at once .

Which two would deliver the most immediate value for your workflow?

Perfect. Let's lock in this architecture and get you operational.

Your Confirmed System Design





Build Sequence

Week Focus Deliverable

- 1 Operations core Executive functions live (email triage, calendar protection, task capture)
- 2 Operations social layer Social media management integrated, unified voice established
- 3 Intelligence foundation Research pipelines running, first automated briefs to Operations

4 Feedback loops Intelligence outputs auto-feed content calendar and email digests

Immediate Next Step

Choose your Operations Agent platform (decides your Week 1 velocity):

OptionSpeed Control Best If

Lindy 2 daysMedium You want live fast, iterate in platform

Relevance AI 3-4 days High You want templates + customization

Custom (CrewAI/LangGraph) 1-2 weeks Full You have dev resources, want to own IP

Microsoft Copilot Studio 3-5 days Medium Deep in Microsoft ecosystem

My recommendation: Start with Lindy or Relevance AI to get operational within 48 hours. Validate the workflow. If you hit platform limits in Month 2, migrate to custom with proven requirements.

What's your technical comfort level—no-code/low-code preference, or do you have development resources available?